

Competitive Programming

Section 3: Counting, frequencies, and maps

Competitive Programming Course

2026-07-12

Table of contents

1	Why counting matters	3
2	Arrays, maps, and coordinate ranges	4
2.1	Frequency array	4
2.2	Map or dictionary	4
3	Choosing the representation	6
4	Preprocessing count queries	7
5	Worked example: value counts	8
6	Tie-breaking	9
6.1	Why map iteration order does not solve ties	9
7	Pair counting	11
8	Worked example: pair sum	12
9	Avoiding double-counting	13
10	Integer sizes	14
11	Hash maps in C++ and Python	15
12	Proof habits	16
13	Common mistakes	17
14	Before you submit	18
15	Exercises	19

1 Why counting matters

Many contest problems become easier when you stop thinking about the input as a list and start thinking about it as counts.

Instead of:

```
1 2 2 3 1
```

think:

```
1 appears 2 times  
2 appears 2 times  
3 appears 1 time
```

The second view is a **frequency table**. It supports questions such as:

- how many times does x appear?
- which value appears most often?
- how many pairs sum to k ?
- can two arrays be made equal by reordering?
- how many distinct values are present?

Frequency tables are one of the first real preprocessing tools in competitive programming: spend linear time once, then answer questions quickly.

2 Arrays, maps, and coordinate ranges

There are two common implementations.

2.1 Frequency array

Use an array when values are small nonnegative integers.

```
values: 0 <= ai <= 1000000  
freq[x] = number of occurrences of x
```

Advantages:

- very fast;
- predictable memory;
- simple indexing.

Disadvantages:

- impossible if values are negative unless shifted;
- wasteful if values can be as large as 10^9 ;
- memory depends on value range, not number of input elements.

2.2 Map or dictionary

Use a map when values are large, negative, sparse, or not known in advance.

In C++:

```
unordered_map<int, int> freq;
```

In Python:

```
from collections import Counter  
freq = Counter(values)
```

Advantages:

- handles negative and large values naturally;
- memory depends mostly on number of distinct values;
- convenient for sparse inputs.

Disadvantages:

- hash-map constants are larger than array indexing;
- C++ `unordered_map` can be vulnerable to adversarial cases in some contests;
- iteration order is not sorted.

3 Choosing the representation

Ask two questions:

1. How many input elements are there?
2. What is the range of possible values?

Value constraint	Good representation
$0 \leq a_i \leq 1000$	frequency array
$-1000 \leq a_i \leq 1000$	shifted frequency array or map
$a_i \leq 10^9$	map/dictionary
strings	map/dictionary
values need sorted order	ordered map or sort keys

If the statement says $n \leq 200000$ and $a_i \leq 10^9$, an array of size $10^9 + 1$ is not acceptable. The number of elements is small; the value range is huge.

4 Preprocessing count queries

Suppose an array is fixed and there are many queries:

```
array: 1 2 2 3 1
queries: 1, 2, 4, 3
```

Without preprocessing, each query scans the whole array:

```
 $O(nq)$ 
```

With a frequency table:

```
build table:  $O(n)$ 
each query:  $O(1)$  average with hash map
total:  $O(n + q)$ 
```

For $n = 200000$ and $q = 200000$, that difference is enormous:

```
nq      = 400000000000
n + q   = 400000
```

This is the standard shape:

```
raw input
  |
  v
preprocess into a summary
  |
  v
answer many queries using the summary
```

5 Worked example: value counts

Array:

```
1 2 2 3 1
```

Build:

value	count
1	2
2	2
3	1

Queries:

query	answer
1	2
2	2
4	0
3	1

The missing value 4 is important. A query may ask for something absent from the map. The answer is 0, not an error.

6 Tie-breaking

Frequency problems often ask for an extreme with a tie rule:

Print the value that appears most often. If several values tie, print the smallest value.

The count alone is not enough. You must compare pairs:

```
better candidate if:  
    count is larger  
    OR count is equal and value is smaller
```

For:

```
-1 -2 -2 -1 -3 -3
```

the table is:

value	count
-1	2
-2	2
-3	2

All counts tie, so the answer is `-3 2`.

6.1 Why map iteration order does not solve ties

Python dictionaries preserve insertion order, and C++ `unordered_map` has an implementation-dependent order. Neither is the tie rule.

For the earlier array, the keys are first encountered in this order:

```
-1, -2, -3
```

If you simply keep the first key whose count is maximal, you print `-1`. That is wrong: the statement asks for the **smallest** tied value, `-3`. Iteration order describes how a container happens to visit keys; it does not describe which key the problem wants.

The safe method is to compare both parts of the answer explicitly:

```
best_value = None
best_count = -1

for value, count in freq.items():
    if count > best_count or (count == best_count and value < best_value):
        best_value = value
        best_count = count
```

The same rule in C++ is:

```
if (count > best_count || (count == best_count && value < best_value)) {
    best_value = value;
    best_count = count;
}
```

Read the comparison as a sentence: choose a larger count; if counts tie, choose the smaller value. If a problem instead asks for the largest tied value, change only the second comparison to `value > best_value`.

7 Pair counting

Now consider:

Count pairs $i < j$ with $a_i + a_j = k$.

The direct brute force is:

```
for i:
    for j > i:
        check  $a_i + a_j$ 
```

That is $O(n^2)$. For $n = 200000$, it is impossible.

The frequency idea is to scan left to right. When you are at value x , a previous value must be:

```
 $k - x$ 
```

If that complement has appeared c times before, then the current position forms c new valid pairs.

Algorithm:

```
seen = empty frequency table
answer = 0
for x in array from left to right:
    answer += seen[k - x]
    seen[x] += 1
```

This counts each pair exactly once: when its right endpoint is processed.

8 Worked example: pair sum

Array:

```
1 4 2 3 5
```

Target:

```
5
```

Trace:

current x	needed	seen before	new pairs	answer
1	4	none	0	0
4	1	one 1	1	1
2	3	none	0	1
3	2	one 2	1	2
5	0	none	0	2

The pairs are (1, 4) and (2, 3) by value, at their actual indices.

For repeated values:

```
array = 1 1 1 1  
target = 2
```

Trace:

index	x	seen[1] before	new pairs	answer
0	1	0	0	0
1	1	1	1	1
2	1	2	2	3
3	1	3	3	6

The answer is 6, which is 4 choose 2.

9 Avoiding double-counting

Another possible method is to build the whole frequency table and combine counts by value. That is valid, but it requires careful handling of:

- x paired with a different value $k - x$;
- x paired with itself when $2x = k$;
- avoiding counting both (x, y) and (y, x) .

The left-to-right method avoids these cases naturally. It is often the simpler contest implementation.

10 Integer sizes

Counts and answers may be larger than individual values.

If `n = 200000`, the number of pairs can be:

$$\binom{200000}{2} = 19999900000.$$

That does not fit in 32-bit `int`. Use `long long` for pair counts in C++.

In Python, integers are arbitrary precision, but you still need to understand why the answer can be large.

11 Hash maps in C++ and Python

For C++ `unordered_map`, it is often useful to reserve space:

```
unordered_map<int, int> freq;  
freq.reserve(2 * n + 1);
```

This reduces rehashing overhead.

For Python:

```
from collections import Counter, defaultdict
```

Use `Counter(values)` when you already have all values. Use `defaultdict(int)` when scanning and updating one value at a time.

12 Proof habits

For count-query preprocessing, the invariant is:

after processing a prefix of the array, **freq**[**x**] equals the number of occurrences of **x** in that prefix.

For pair counting, the invariant is:

before processing position **j**, **seen**[**x**] equals the number of positions **i** < **j** with value **x**, and **answer** equals the number of valid pairs fully contained in the processed prefix.

When the current value is **a_j**, every new pair ending at **j** needs a previous value **k - a_j**. There are exactly **seen**[**k - a_j**] such positions.

13 Common mistakes

- Using a frequency array when values can be 10^9 .
- Forgetting negative values.
- Returning an arbitrary tied value instead of applying the tie rule.
- Treating absent map keys as errors instead of count zero.
- Counting pairs twice.
- Counting (i, i) as a pair.
- Using 32-bit `int` for a pair-count answer.
- Assuming hash-map iteration order is sorted.

14 Before you submit

For every frequency problem, ask:

1. Are values small enough for an array, or do I need a map?
2. Are values negative?
3. What should happen for absent values?
4. Is there a tie-breaking rule?
5. Does the answer count values, indices, pairs, or distinct values?
6. Can the count exceed 32-bit range?

15 Exercises

The local exercises focus on three core frequency patterns:

1. `a_value_counts`: preprocess counts and answer queries.
2. `b_frequency_winner`: choose an extreme with a tie rule.
3. `c_pair_sum_count`: count pairs by scanning previous complements.

Run:

```
python3 sections/03_counting_frequencies/check.py
```